ESPECIFICAÇÃO SDD — TASKANALYZER

Fase 1 — Especificação e Governança de Contexto (SDD & AI Harness)

Bootcamp III — Entrega Inicial (Atividade Cumulativa — Fase 1 de 3)

1. Visão Geral e Contrato de Negócio (SDD)

1.1 Identificação

Campo	Valor
Nome completo	Juan Conceição Cassimiro
Curso	Ciências da Computação
Polo/Turma	Turma A - 0726
E-mail institucional	juan.cassimiro67@sempreceub.com
Data de elaboração	24/08/2026
Versão da especificação	1.0

1.2 Propósito do Módulo TaskAnalyzer

O módulo TaskAnalyzer é um componente de análise de tarefas e produtividade que recebe um conjunto de tarefas de um usuário ou equipe e calcula métricas objetivas de desempenho a partir dos dados de criação, execução e conclusão dessas tarefas.

O escopo funcional do módulo compreende:

- Cálculo do tempo médio de conclusão das tarefas finalizadas, considerando o intervalo entre o início e a conclusão da execução.
- Cálculo da taxa de atraso, isto é, o percentual de tarefas concluídas após o prazo estabelecido.
- Geração de indicadores agregados por prioridade (baixa, média, alta), permitindo identificar gargalos e padrões de desempenho por categoria de tarefa.

O módulo não realiza persistência de dados, não possui interface gráfica e não se comunica com serviços externos — atua exclusivamente como uma camada de análise que recebe uma lista de tarefas em memória e devolve um conjunto de métricas calculadas.

1.3 Contrato Executável de Interface

Este contrato formaliza as entradas, saídas, tipos de dados e restrições que o código gerado pela IA deverá seguir estritamente na Fase 2. Ele é a fonte da verdade para a implementação.

1.3.1 Entradas (Inputs)

Cada tarefa processada pelo TaskAnalyzer é representada pelos seguintes campos:

Campo	Tipo	Obrigatório	Descrição / Restrições
id_tarefa	int	Sim	Identificador único da tarefa (> 0).
titulo	str	Sim	Título/descrição curta da tarefa.
prioridade	str (enum)	Sim	Valores aceitos: "baixa", "média", "alta".
status	str (enum)	Sim	Valores aceitos: "concluída", "pendente", "cancelada".
data_criacao	datetime	Sim	Data/hora de criação da tarefa (UTC).

Campo	Tipo	Obrigatório	Descrição / Restrições
data_inicio	datetime	Não	Data/hora de início da execução (UTC).
data_conclusao	datetime	Não	Data/hora de conclusão (UTC). Obrigatória quando status = "concluída".
prazo	datetime	Sim	Data-limite prevista para conclusão (UTC).

1.3.2 Saídas (Outputs)

A execução do TaskAnalyzer sobre uma lista de tarefas deve retornar a seguinte estrutura de métricas:

Campo	Tipo	Descrição
quantidade_tarefas	int	Total de tarefas analisadas no conjunto de entrada.
tempo_medio_conclusao_horas	float	Tempo médio (em horas) entre data_inicio e data_conclusao das tarefas concluídas.
taxa_atraso_percentual	float	Percentual de tarefas concluídas cuja data_conclusao é posterior ao prazo.
indicadores_por_prioridade	dict	Dicionário com as métricas acima recalculadas para cada prioridade (baixa, média, alta).

1.3.3 Regras de Negócio e Restrições

- Apenas tarefas com status "concluída" entram no cálculo de tempo_medio_conclusao_horas.
- Tempo de conclusão = data_conclusao − data_inicio, expresso em horas, arredondado com 2 casas decimais.
- Uma tarefa é considerada em atraso quando data_conclusao > prazo.
- Tarefas com datas inválidas (ex.: data_conclusao anterior a data_inicio, ou data_inicio posterior a data_conclusao) não devem ser silenciosamente ignoradas: devem disparar uma exceção específica e tratada.
- Situações de divisão por zero (ex.: nenhuma tarefa concluída no conjunto) devem ser tratadas explicitamente, retornando 0.0 ou lançando exceção específica, conforme definido nos cenários de aceite.
- Todas as datas devem ser interpretadas e comparadas em UTC.
- Percentuais e médias devem ser arredondados para 2 casas decimais.

2. Especificação de Cenários de Aceite e Test Harness

2.1 Cenários de Aceite (Dado–Quando–Então)

Cenário 1 — Sucesso

- Dado: um conjunto de tarefas válidas, com prioridades, prazos e status corretamente preenchidos.
- Quando: o TaskAnalyzer for executado sobre esse conjunto.
- Então: o sistema deve retornar as métricas corretas — tempo médio de conclusão, taxa de atraso e quantidade de tarefas — corretamente segmentadas por prioridade e de forma geral.

Cenário 2 — Exceção / Erro

- Dado: uma entrada contendo datas inválidas (ex.: data_conclusao anterior à data_inicio) ou uma situação de divisão por zero (nenhuma tarefa concluída no período).
- Quando: o TaskAnalyzer for executado sobre esse conjunto.
- Então: o sistema deve disparar uma exceção específica e tratada, com mensagem clara e objetiva, sem interromper o programa de forma não controlada.

2.2 Planejamento do Test Harness

Os cenários acima serão transformados em testes automatizados na Fase 2, seguindo o fluxo:

- Cenários de Aceite (este documento) — descrevem o comportamento esperado do sistema e servem como referência única de verdade.
- Casos de Teste — decomposição de cada cenário em casos concretos, com dados de entrada específicos e resultado esperado explícito (ex.: 3 tarefas concluídas, 1 em atraso → taxa_atraso_percentual = 33.33).
- Testes Automatizados (pytest) — implementação dos casos de teste usando fixtures para preparar os dados de entrada e assertions para validar outputs, tipos e exceções levantadas.
- Execução e Validação contínua — os testes serão executados automaticamente a cada geração/alteração de código pela IA; qualquer falha bloqueia a homologação.
- Relatórios e Evidências — geração de relatório de cobertura e resultados, garantindo rastreabilidade entre cada cenário de aceite e o respectivo teste automatizado.

3. Governança de Contexto e Regras para Agentes de IA (CONTEXT_RULES)

As regras abaixo comporão o arquivo CONTEXT_RULES.md, que será a fonte de verdade fornecida aos assistentes de IA na Fase 2, garantindo alinhamento, consistência e qualidade do código gerado.

3.1 Diretrizes Arquiteturais (Obrigatórias)

- Python 3.11+ como versão mínima obrigatória.
- Utilizar type hints (anotações de tipo) em todas as funções e métodos.
- Seguir o princípio Single Responsibility (SRP) e código limpo (PEP 8).
- Documentação formal no padrão Google style docstrings em módulos, classes, funções e parâmetros.
- Funções devem ser pequenas, coesas e com um único propósito.
- Tratamento de exceções específico e com mensagens claras.
- Logs estruturados utilizando o módulo padrão logging.
- Código legível, manutenível e com nomes de variáveis e funções descritivos.

3.2 Proibições Explícitas (Regras Restritivas)

- Não utilizar bibliotecas externas não autorizadas.
- Não alterar, remover ou adaptar os cenários de teste fornecidos.
- Não modificar a estrutura de pastas definida nesta especificação.
- Não persistir dados em arquivos ou bancos de dados.
- Não alterar a assinatura (nome e parâmetros) das funções públicas sem autorização.
- Não gerar código sem testes correspondentes para as novas funcionalidades.
- Não inserir código duplicado ou sem necessidade.
- Não assumir comportamentos não especificados no contrato de negócio.

3.3 Regras de Interação com a IA

- Fornecer sempre o contexto completo (especificação, cenários e regras) antes de solicitar geração de código.
- Validar se a IA compreendeu o contrato antes de aceitar a solução proposta.
- Solicitar explicações à IA sempre que houver ambiguidade na solução apresentada.
- Revisar criticamente todo código gerado antes de qualquer commit.
- Rejeitar e solicitar nova geração sempre que a resposta violar as proibições ou o contrato definido.

4. Arquitetura do Repositório e Preparação para Git/GitHub

4.1 Desenho da Árvore do Repositório

Estrutura de arquivos que será versionada no GitHub a partir da Fase 2:

```
sdd-task-analyzer/
├── README.md              # Visão geral do projeto no GitHub
├── CONTEXT_RULES.md       # Regras persistentes para a IA
├── specs/
│   └── task_analyzer_spec.md   # Especificação SDD (baseada nesta Fase 1)
├── tests/
│   └── test_harness.py   # Testes automatizados de validação (pytest)
├── src/
│   └── task_analyzer.py  # Código gerado via IA e homologado
├── .gitignore
└── requirements.txt
```

4.2 Plano de Homologação Humana

Na Fase 2, todo código gerado pela IA será submetido a uma revisão crítica humana antes de ser aceito e versionado. A homologação seguirá os passos:

1. Execução completa dos testes automatizados para validação funcional.

2. Revisão de conformidade com o contrato de negócio e as CONTEXT_RULES.

3. Análise de qualidade de código, legibilidade, tratamento de erros e aderência aos padrões definidos.

4. Testes manuais complementares em casos de borda.

5. Somente após aprovação, o código será commitado no repositório.

Esse processo garante que a IA seja uma ferramenta produtiva, mas que a decisão final e a responsabilidade técnica permaneçam humanas.

Código de Ética da EaD: mantenho os princípios de autoria individual na elaboração deste trabalho e da gravação do vídeo de apresentação.